

EQREV DATA ENGINEER HIRING ASSIGNMENT

Practical Assessment

1. API Data Fetch (TataCliqu)

Target Website:

Category URL: <https://www.tatacliq.com/casual-wear/c-msh1116>

API Endpoint: <https://searchbff.tatacliq.com/products/mpl/search> (find the actual endpoint)

Category ID: msh1116

Objective:

Build a Python/Javascript code to fetch structured product data using the TataCliqu API.

Requirements:

- Identify correct API parameters using browser DevTools
- Fetch at least 20–30 products
- Extract:
 - Product Name
 - Brand
 - Original Price
 - Discount Price
 - Discount Percentage
 - Rating
 - Number of Reviews
 - Product ID or URL
- Handle pagination
- Handle network failures and missing fields gracefully
- Save output to CSV (structured format)

2. API Interception using Playwright (MaxFashion)

Target Website:

Category URL: <https://www.maxfashion.in/in/en/c/maxmen-tops-polos?p=1>

Objective:

Use Playwright to intercept product API calls made by the website and extract structured JSON data.

Requirements:

- Capture API responses via network interception
- Extract:
 - Product Name
 - Brand (if available)

- Original Price
- Discount Price
- Discount Percentage
- Product Image URL
- Product ID / SKU
- Color / Variant
- Handle at least 2–3 pages
- Save structured output to CSV
- Implement proper error handling

Focus on structured API extraction instead of HTML scraping.

3. Build a Simple REST Data Service

Objective:

Build a REST API (Flask / FastAPI / Express) implementing CRUD operations.

Required Endpoints:

- GET /items
- POST /items
- GET /items/:id
- PUT /items/:id
- DELETE /items/:id

Requirements:

- Store data in memory
- Proper HTTP status codes
- Input validation
- Clear JSON response structure
- Data caching (bonus)

4. Containerization with Docker

Objective:

Create a Dockerfile for one of your applications.

Requirements:

- Use appropriate base image
- Install dependencies
- Copy application code
- Run application using CMD
- Add .dockerignore

Bonus: Multi-stage build, non-root user, docker-compose.yml

5. Exercise 5 - CI/CD with GitHub Actions

Objective:

Create a GitHub Actions workflow.

Requirements:

- Trigger on push and pull request
- Install dependencies
- Run lint checks
- Execute tests (if applicable)

Bonus: Matrix builds, dependency caching, coverage upload

Submission Guidelines

Submit:

- Source code files
- README.md with setup instructions
- requirements.txt or package.json
- Any Docker / CI configuration files

Total Time: 2.5 Hours

We value clean, reliable, production oriented data pipelines.

Best of luck!